

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 3

Выполнил:

Смирнова Карина

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2022 г.

Задача

Задание:

- реализовать тестирование API средствами Postman;
- написать тесты внутри Postman.

В результате хочу получить один комплексный сценарий по основному процессу в вашем приложении.

Пример такого сценарий для системы по аренде недвижимости: регистрация -> вход -> список объявлений -> фильтрация по списку -> просмотр случайного объявления из списка -> старт переписки по объявлению.

Ход работы

Для реализации автотестов и обеспечения непрерывности сценария была настроена коллекция **RentingServiceCenarioTest** и созданы переменные окружения, позволяющие динамически передавать данные (токены, идентификаторы объектов, чатов и бронирований) между запросами без ручного вмешательства.

Описание переменных окружения

Для автоматизации сценария используются следующие ключевые переменные:

- baseUrl: Хост тестируемого API.
- accessToken / refreshToken: Авторизационные токены пользователя.
- userId, email, password: Данные учетной записи тестируемого пользователя.
- listingId: Идентификатор созданного пользователем объекта недвижимости.
- forRentId: Идентификатор выбранной для аренды квартиры (запоминается из общего списка).
- bookingId: Идентификатор созданной заявки на бронирование.
- chatId, messageId: Переменные для проверки работы модуля чатов.

Сценарий тестирования и логика проверок

Коллекция состоит из 13 последовательных шагов, имитирующих реальный пользовательский путь и проверяющих бизнес-логику системы (как позитивные, так и негативные кейсы).

1. Модуль аутентификации и профиля пользователя

- POST 1 - Registration: Регистрация нового пользователя.
 - *Логика теста:* Проверка успешного создания аккаунта (HTTP-статус 201 Created). Переменные email и password генерируются динамически и сохраняются в окружении.
- POST 1.1 Fail Registration (Негативный): Повторная попытка регистрации с тем же email.
 - *Логика теста:* Проверка валидации уникальности почты. Ожидается ошибка (409 Conflict).
- POST 2 - Login: Авторизация пользователя в системе.
 - *Логика теста:* Проверка успешного входа. Скрипт во вкладке Tests автоматически извлекает полученные accessToken и refreshToken и сохраняет их в переменные окружения для последующих запросов.

2. Модуль работы с объявлениями (Listings)

- POST 3 - Create Listing: Создание объявления об аренде недвижимости.
 - *Логика теста:* Проверяется успешность публикации. Полученный id созданного объекта сохраняется в переменную listingId.
- GET 4 - Get Listings: Получение общего списка доступных объявлений.
 - *Логика теста:* Тест проверяет, что созданное на предыдущем шаге объявление появилось в общей базе.
- GET 5 - Listings Radius: Фильтрация объявлений по радиусу (геолокации).
 - *Логика теста:* Проверка корректности работы гео-фильтра. Запрос отправляет координаты, тест проверяет, что в ответе возвращаются только объекты в заданном радиусе.

- GET 6 - Get Listings Monthly: Фильтрация по типу аренды (MONTHLY - ежемесячно).
 - *Логика теста:* Проверяется работа фильтра по типу коммерческих условий. Ответ должен содержать только долгосрочные объявления. Из полученного списка скриптом выбирается и сохраняется первая доступная чужая недвижимость (forRentId) для последующего бронирования.

3. Модуль бронирования (Booking)

- POST 7 - Do Rent: Создание заявки на бронирование объекта forRentId.
 - *Логика теста:* Проверка позитивного сценария аренды. Ответ должен содержать статус бронирования, а динамический bookingId сохраняется в окружение.
- POST 7.1 Double Booking Not Work (Негативный): Попытка повторного бронирования того же объекта.
 - *Логика теста:* Проверка защиты от овербукинга. Система должна отклонить запрос без ошибки, не создавая дубликат брони.
- POST 7.2 - Booking Owner Fail (Негативный): Попытка пользователя забронировать собственную недвижимость (listingId).
 - *Логика теста:* Проверка бизнес-правила. Система должна вернуть ошибку валидации, так как владелец не может арендовать сам у себя.

4. Модуль чатов и коммуникации

- GET 8 - Chat Created Auto Check: Проверка автоматического создания чата.
 - *Логика теста:* Бизнес-логика предполагает, что при бронировании между арендатором и владельцем автоматически создается диалог. Тест проверяет наличие нового чата и сохраняет chatId.
- POST 9 - Send Message In Chat: Отправка сообщения в созданный чат.

- *Логика теста:* Проверка работоспособности мессенджера внутри платформы. Сообщение отправляется с использованием chatId, сохраняется messageId.
- GET 10 - Check Message Is In Chat: Проверка доставки и отображения сообщения.
 - *Логика теста:* Финальная точка сценария. Запрос получает историю сообщений чата, тест проверяет, что отправленное на шаге 9 сообщение присутствует в теле ответа и отображается корректно.

Вывод

В ходе выполнения работы был успешно настроен сквозной автоматический сценарий в Postman. Благодаря использованию вкладок Pre-request Script и Tests удалось полностью исключить ручной ввод данных между запросами. Коллекция успешно тестирует не только стандартное поведение системы, но и критические бизнес-ограничения.